

Introduction à la programmation en Python

Variables, boucles, fonctions — avec l'IDE Thonny

Seconde, Première, Terminale

Contents

1	Découvrir Python et l'IDE Thonny	3
1.1	Pourquoi programmer en mathématiques ?	3
1.2	Qu'est-ce qu'un IDE ? Présentation de Thonny	3
2	Les bases du langage	5
2.1	Qu'est-ce qu'une variable en informatique ?	5
2.2	Les types de données de base	5
2.3	Opérateurs et entrées-sorties	6
3	Structures conditionnelles	8
4	Les boucles	9
4.1	La boucle <code>for</code> : répéter un nombre connu de fois	9
4.2	La boucle <code>while</code> : répéter jusqu'à une condition	10
4.3	Comparaison <code>for</code> / <code>while</code>	11
5	Écrire ses propres fonctions	12
5.1	Pourquoi créer une fonction ?	12
6	Fiche mémo	14
7	Exercices	15
7.1	Niveau Seconde — variables, types, conditions	15
7.2	Niveau Première — boucles <code>for</code>	15
7.3	Niveau Terminale — boucles <code>while</code> et fonctions	16

Apprendre à coder n'est une mince affaire pour personne. La lecture de ce papier devra toujours se faire accompagnée de l'IDE présenté par suite, ainsi qu'une page web permettant de faire des recherches approfondies sur les opérateurs ou autres fonctionnalités mentionnés. On code sur un ordinateur, pas un téléphone ni une tablette.

Chapter 1

Découvrir Python et l'IDE Thonny

1.1 Pourquoi programmer en mathématiques ?

Remarque 1.1.1. *Au lycée, la programmation n'est pas une matière séparée des mathématiques : c'est un **outil**. Elle permet de tester une conjecture sur de nombreuses valeurs, de simuler une expérience aléatoire des milliers de fois, d'approcher numériquement une solution qu'on ne sait pas calculer exactement (par dichotomie par exemple), ou encore de traduire un algorithme décrit en français en instructions exécutables par une machine. Le langage utilisé dans l'ensemble du programme de mathématiques (et de spécialité NSI) est **Python**, un langage réputé pour sa syntaxe proche du langage naturel.*

1.2 Qu'est-ce qu'un IDE ? Présentation de Thonny

Définition 1.2.1 (IDE). *Un **IDE** (Integrated Development Environment, environnement de développement intégré) est un logiciel qui regroupe, dans une seule fenêtre, tous les outils nécessaires pour écrire, exécuter et corriger un programme :*

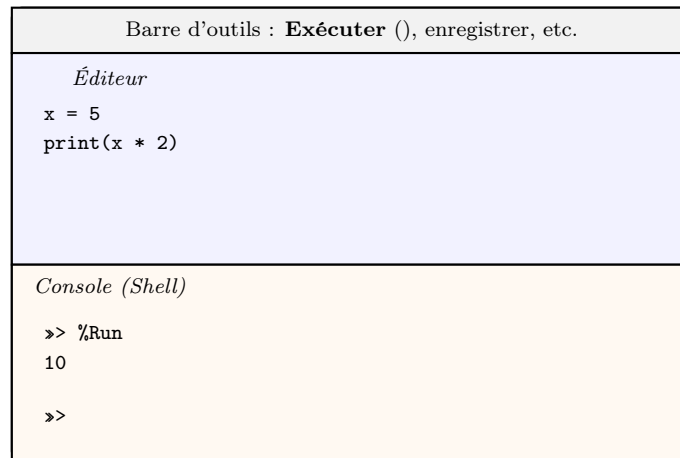
- *un **éditeur de texte** pour écrire le code source (avec coloration syntaxique) ;*
- *une **console** (ou shell), qui exécute le programme et affiche ses résultats, et permet aussi de taper des instructions Python directement, une par une ;*
- *un **interpréteur Python**, le programme qui lit le code écrit et l'exécute réellement ;*
- *des outils d'aide au débogage (repérage et correction des erreurs).*

Sans IDE, il faudrait écrire son code dans un simple bloc-notes, puis l'exécuter séparément en ligne de commande : l'IDE réunit tout cela pour simplifier le travail, en particulier pour un débutant.

Méthode 1.2.2 (Installer Thonny). *Thonny est l'IDE recommandé pour débuter en Python : il est **gratuit, disponible sur Windows, macOS et Linux**, et surtout conçu **spécifiquement pour l'apprentissage** (contrairement à des IDE plus complexes comme PyCharm ou VS Code, qui sont davantage destinés à des développeurs déjà expérimentés). Il a l'avantage énorme d'intégrer directement l'interpréteur Python : il n'y a **rien d'autre à installer**.*

- 1) *Se rendre sur le site officiel : <https://thonny.org> ;*

- 2) Télécharger l'installateur correspondant à son système d'exploitation (Windows / macOS / Linux) ;
- 3) Lancer l'installateur et suivre les instructions par défaut ;
- 4) Ouvrir Thonny : une fenêtre s'affiche, divisée en deux zones principales (l'éditeur en haut, la console en bas).



- Méthode 1.2.3** (Écrire et exécuter son premier programme). 1) Dans la zone **éditeur** (en haut), taper les lignes de code du programme ;
- 2) Enregistrer le fichier (**Ctrl+S**), avec l'extension **.py** ;
 - 3) Cliquer sur le bouton **Exécuter** (la flèche verte), ou appuyer sur la touche **F5** ;
 - 4) Les résultats (affichages, erreurs éventuelles) apparaissent dans la **console** (en bas).

Remarque 1.2.4. La console peut aussi être utilisée seule, en **mode interactif** : on tape une instruction après l'invite `»>` et on obtient le résultat immédiatement, sans avoir à écrire un fichier complet. C'est très pratique pour tester rapidement une ligne de calcul.

Chapter 2

Les bases du langage

2.1 Qu'est-ce qu'une variable en informatique ?

Définition 2.1.1 (Variable). Une **variable** est un espace de la mémoire de l'ordinateur auquel on donne un **nom**, et dans lequel on peut stocker une **valeur**. Contrairement à la notion mathématique de variable (qui désigne souvent un nombre inconnu ou variable dans une expression), une variable informatique est concrètement une **étiquette posée sur une case mémoire** : elle contient une valeur précise à un instant donné, et cette valeur peut être **modifiée** au cours de l'exécution du programme.

Méthode 2.1.2 (Créer une variable : l'affectation). En Python, on crée (ou on modifie) une variable à l'aide du symbole `=`, appelé opérateur d'**affectation** :

```
1 nom_de_la_variable = valeur
```

Cette instruction se lit la variable `nom_de_la_variable` **reçoit** la valeur `valeur` — et surtout **pas** est égale à `=`, car l'affectation n'est **pas** symétrique (voir la remarque ci-dessous).

Exemple 2.1.3.

```
1 age = 17
2 prenom = "Camille"
3 moyenne = 14.5
4 print(age)
5 print(prenom)
```

Ici, la variable `age` contient l'entier 17, `prenom` contient la chaîne de caractères "Camille", et `moyenne` contient le nombre décimal 14,5.

Piège classique. En Python, `x = x + 1` n'est **pas** une égalité mathématique absurde : c'est une instruction qui signifie calculer la valeur actuelle de `x`, lui ajouter 1, puis stocker le résultat dans `x` (qui **écrase** l'ancienne valeur) . Après cette ligne, `x` a changé de valeur : ce n'est pas une propriété permanente, contrairement à une égalité en mathématiques.

2.2 Les types de données de base

Définition 2.2.1 (Type d'une variable). *Chaque valeur manipulée par Python possède un **type**, qui détermine la nature de la donnée stockée et les opérations qu'on peut effectuer dessus. On peut connaître le type d'une variable avec la fonction `type()`.*

Type	Signification	Exemple
int	Nombre entier	x = 12
float	Nombre décimal (<i>floating point</i>)	x = 3.14
str	Chaîne de caractères (<i>string</i>)	x = "bonjour"
bool	Booléen (vrai/faux)	x = True
list	Liste de valeurs	x = [1, 2, 3]

Exemple 2.2.2.

```

1 x = 7
2 y = 7.0
3 z = "7"
4 print(type(x)) # <class 'int'>
5 print(type(y)) # <class 'float'>
6 print(type(z)) # <class 'str'>

```

Ces trois variables contiennent des données qui se ressemblent mais sont de types différents : 7 n'est pas la même chose que 7,0, ni que la chaîne de caractères "7" (qu'on ne peut pas directement additionner à un nombre).

2.3 Opérateurs et entrées-sorties

Proposition 2.3.1 (Opérateurs arithmétiques usuels).

Opérateur	Signification	Exemple
+ - *	addition, soustraction, multiplication	3 * 4 donne 12
/	division (résultat toujours en float)	7 / 2 donne 3.5
//	division entière (quotient)	7 // 2 donne 3
%	modulo (reste de la division)	7 % 2 donne 1
**	puissance	2 ** 3 donne 8

Méthode 2.3.1 (Communiquer avec l'utilisateur). *Deux fonctions permettent au programme d'interagir avec l'utilisateur :*

- `print(...)` **affiche** une valeur ou un message dans la console ;
- `input("message")` **demande** une saisie à l'utilisateur, et **renvoie toujours une chaîne de caractères** (**str**), même si l'utilisateur tape un nombre — il faut donc la convertir avec `int(...)` ou `float(...)` si l'on veut faire des calculs.

Exemple 2.3.2.

```

1 nom = input("Quel est ton prenom ? ")
2 age = int(input("Quel est ton age ? "))

```

```
3 print("Bonjour", nom, ", tu as", age, "ans.")  
4 print("Dans 10 ans tu auras", age + 10, "ans.")
```


Chapter 3

Structures conditionnelles

Remarque 3.0.1. Avant de pouvoir écrire des boucles utiles, il faut savoir faire faire à un programme des choix *différents* selon une condition : c'est le rôle des structures conditionnelles `if` / `elif` / `else`.

Définition 3.0.2 (Expression booléenne). Une *expression booléenne* est une expression dont la valeur est soit *True* (vrai), soit *False* (faux). On les construit avec les opérateurs de comparaison (`==`, `!=`, `<`, `<=`, `>`, `>=`) et les connecteurs logiques (`and`, `or`, `not`).

Piège classique. Le symbole `==` sert à **tester une égalité** (il renvoie `True` ou `False`), alors que `=` sert à **affecter** une valeur. Confondre les deux est l'une des erreurs les plus fréquentes chez les débutants.

Méthode 3.0.3 (Structure `if` / `elif` / `else`).

```
1 if condition_1:  
2     # instructions executees si condition_1 est vraie  
3 elif condition_2:  
4     # instructions executees si condition_1 est fausse et condition_2 vraie  
5 else:  
6     # instructions executees si aucune condition n'est verifiee
```

En Python, l'*indentation* (les espaces en début de ligne) n'est pas une simple mise en forme : elle indique quelles instructions appartiennent à quel bloc. C'est une contrainte du langage, pas une option de style.

Exemple 3.0.4.

```
1 note = float(input("Entrez une note sur 20 : "))  
2 if note >= 10:  
3     print("Note validee.")  
4 else:  
5     print("Note non validee.")
```

Chapter 4

Les boucles

4.1 La boucle for : répéter un nombre connu de fois

Remarque 4.1.1 (Besoin). *On utilise une boucle `for` lorsqu'on connaît à l'avance le nombre de répétitions à effectuer (par exemple : calculer les 10 premiers termes d'une suite, parcourir tous les éléments d'une liste, répéter une simulation 1000 fois).*

Méthode 4.1.2 (Boucle for avec range).

```
1 for i in range(n):
2     # instructions repetees, avec i qui prend
3     # successivement les valeurs 0, 1, 2, ..., n-1
```

La fonction `range(n)` génère les entiers de 0 à $n - 1$ (soit n valeurs). On peut aussi utiliser `range(a, b)` (entiers de a à $b - 1$) ou `range(a, b, pas)` (avec un pas différent de 1).

Exemple 4.1.3.

```
1 for i in range(5):
2     print(i)
3 # Affiche : 0 1 2 3 4
```

Exemple 4.1.4 (Calcul d'une somme). *Calculons la somme des n premiers entiers (avec n donné) :*

```
1 n = 10
2 somme = 0
3 for i in range(1, n + 1):
4     somme = somme + i
5 print(somme) # Affiche 55
```

On retrouve ici un **schéma algorithmique fondamental** : la variable `somme`, appelée **accumulateur**, est initialisée avant la boucle, puis mise à jour à chaque tour.

Exemple 4.1.5 (Parcourir une liste). *La boucle `for` permet aussi de parcourir directement les éléments d'une liste, sans passer par les indices :*

```
1 notes = [12, 15, 8, 17, 10]
2 total = 0
3 for note in notes:
4     total = total + note
5 moyenne = total / len(notes)
6 print(moyenne)
```

(la fonction `len(...)` renvoie le nombre d'éléments d'une liste.)

4.2 La boucle `while` : répéter jusqu'à une condition

Remarque 4.2.1 (Besoin). On utilise une boucle `while` lorsqu'on **ne connaît pas à l'avance** le nombre de répétitions nécessaires : on répète **tant qu'une condition** reste vraie (par exemple : faire tourner un algorithme jusqu'à obtenir une précision donnée, ou jusqu'à ce qu'une suite dépasse un certain seuil).

Méthode 4.2.2 (Boucle `while`).

```
1 while condition:
2     # instructions repetees tant que condition est vraie
```

Piège classique : la boucle infinie. Il faut impérativement que la condition finisse par devenir fausse, sinon le programme **ne s'arrête jamais**. Cela signifie qu'au moins une variable intervenant dans la condition doit être **modifiée à l'intérieur** de la boucle.

Exemple 4.2.3 (Seuil d'une suite). Déterminons le plus petit rang n à partir duquel la suite $u_n = 1,05^n$ dépasse 2 :

```
1 u = 1
2 n = 0
3 while u <= 2:
4     u = u * 1.05
5     n = n + 1
6 print("Le premier rang est n =", n)
```

Ici, impossible de savoir à l'avance combien de tours seront nécessaires : une boucle `while` est donc indispensable (une boucle `for` ne conviendrait pas).

Exemple 4.2.4 (Dichotomie — rappel). La méthode de dichotomie, vue en cours sur le T.V.I., est un exemple typique d'utilisation de `while`, puisqu'on répète jusqu'à obtenir la précision voulue :

```
1 def dichotomie(f, a, b, precision):
2     while (b - a) > precision:
3         m = (a + b) / 2
4         if f(a) * f(m) <= 0:
5             b = m
6         else:
7             a = m
8     return (a + b) / 2
```

4.3 Comparaison `for` / `while`

	Utiliser <code>for</code>	Utiliser <code>while</code>
Quand ?	Le nombre de répétitions est connu à l'avance	Le nombre de répétitions dépend d'une condition
Risque	Aucune boucle infinie possible (bornée par range)	Boucle infinie si la condition ne devient jamais fausse
Exemples	Parcourir une liste, calculer les n premiers termes	Dichotomie, seuil d'une suite, algorithme d'Euclide

Chapter 5

Écrire ses propres fonctions

5.1 Pourquoi créer une fonction ?

Remarque 5.1.1 (Besoin). *Dès qu'un même calcul (ou une même suite d'instructions) doit être réutilisé à plusieurs endroits d'un programme — ou simplement pour rendre le code plus lisible et mieux organisé — on définit une fonction : on lui donne un nom une seule fois, puis on l'appelle autant de fois que nécessaire, avec des valeurs (les **paramètres**) potentiellement différentes à chaque appel.*

Méthode 5.1.2 (Définir une fonction).

```
1 def nom_fonction(parametre_1, parametre_2):
2     # instructions utilisant les parametres
3     resultat = ...
4     return resultat
```

def introduit la définition (elle ne fait **qu'enregistrer** la fonction, elle ne l'exécute pas) ;

- les **paramètres** sont des variables locales à la fonction, qui reçoivent les valeurs fournies lors de l'appel (les **arguments**) ;
- **return** indique la valeur **renvoyée** par la fonction, et **met fin** à son exécution.

Exemple 5.1.3.

```
1 def carre(x):
2     return x * x
3
4 def moyenne(a, b):
5     return (a + b) / 2
6
7 print(carre(5))           # Affiche 25
8 print(moyenne(12, 16))   # Affiche 14.0
```

Piège classique. Une fonction avec **return** renvoie une valeur utilisable ensuite (`x = carre(5)` stocke 25 dans `x`), alors qu'une fonction qui se contente d'un `print(...)` **affiche** un résultat à l'écran mais **ne renvoie rien** d'exploitable par la suite (elle renvoie en réalité la valeur spéciale `None`). Ne pas confondre afficher et renvoyer .

Exemple 5.1.4 (Fonction utilisant une boucle). *Une fonction qui teste si un entier n est premier (illustration du lien entre fonctions et boucles) :*

```
1 def est_premier(n):
2     if n < 2:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False
7     return True
8
9 print(est_premier(17)) # True
10 print(est_premier(18)) # False
```

Remarque 5.1.5 (Portée des variables). Une variable définie **à l'intérieur** d'une fonction (variable **locale**) n'existe que pendant l'exécution de cette fonction, et n'est pas accessible depuis l'extérieur. C'est une différence essentielle avec une variable définie en dehors de toute fonction (variable **globale**).

Chapter 6

Fiche mémo

Notion	À retenir
Variable	<code>nom = valeur</code> : case mémoire nommée, dont le contenu peut changer.
Types de base	<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code> , <code>list</code> — obtenus avec <code>type(...)</code> .
Entrées-sorties	<code>print(...)</code> affiche ; <code>input(...)</code> renvoie toujours une chaîne (<code>str</code>), à convertir si besoin.
Condition	<code>if ... elif ... else :</code> , indentation obligatoire, <code>==</code> pour tester (<code>=</code> pour affecter).
Boucle <code>for</code>	Nombre de tours connu : <code>for i in range(n):</code> .
Boucle <code>while</code>	Nombre de tours inconnu , dépend d'une condition : <code>while condition:</code> — attention aux boucles infinies.
Fonction	<code>def nom(param): ... return valeur</code> — regroupe et réutilise un traitement.

Chapter 7

Exercices

7.1 Niveau Seconde — variables, types, conditions

Exercice 7.1.1. Écrire un programme qui demande à l'utilisateur son âge (avec `input`), puis affiche *Tu es majeur* si l'âge est supérieur ou égal à 18, et *Tu es mineur* sinon.

Exercice 7.1.2. Écrire un programme qui demande deux nombres `a` et `b`, puis affiche leur somme, leur produit et leur quotient (en gérant le cas où `b` vaut 0).

Exercice 7.1.3. Écrire une fonction `signe(x)` qui renvoie la chaîne `"positif"`, `"negatif"` ou `"nul"` selon le signe du réel `x` passé en paramètre.

Exercice 7.1.4. Écrire un programme qui demande trois notes à l'utilisateur, calcule leur moyenne, et affiche *Admis* si la moyenne est supérieure ou égale à 10, sinon *Non admis*.

Exercice 7.1.5. Sans exécuter le code, prévoir ce qu'affiche le programme suivant, puis vérifier avec Thonny :

```
1 a = 3
2 b = a
3 a = a + 2
4 print(a, b)
```

Expliquer pourquoi `b` n'a **pas** changé de valeur en même temps que `a`.

7.2 Niveau Première — boucles for

Exercice 7.2.1. Écrire un programme qui affiche, à l'aide d'une boucle `for`, tous les entiers pairs de 0 à 20.

Exercice 7.2.2. Écrire une fonction `factorielle(n)` qui calcule $n! = 1 \times 2 \times \dots \times n$ à l'aide d'une boucle `for` et d'un accumulateur (on posera $0! = 1$).

Exercice 7.2.3. Une suite (u_n) est définie par $u_0 = 1$ et $u_{n+1} = 3u_n - 2$ pour tout $n \geq 0$. Écrire une fonction `terme(n)` qui renvoie la valeur de u_n , en utilisant une boucle `for`.

Exercice 7.2.4. Écrire un programme qui demande un entier n , puis calcule et affiche la somme des carrés des entiers de 1 à n (c'est-à-dire $1^2 + 2^2 + \dots + n^2$).

Exercice 7.2.5 (Simulation). On lance 1000 fois un dé à 6 faces à l'aide du module `random` (fonction `random.randint(1, 6)`), et l'on compte combien de fois on obtient un 6. Compléter le programme suivant, puis l'exécuter :

```
1 import random
2
3
4 compteur = 0
5 for i in range(1000):
6     de = random.randint(1, 6)
7     # compléter : incrementer compteur si de vaut 6
8
9 print(compteur)
```

7.3 Niveau Terminale — boucles `while` et fonctions

Exercice 7.3.1. Écrire une fonction `seuil(u0, r, S)` qui renvoie le plus petit entier n tel que le terme u_n d'une suite géométrique de premier terme u_0 et de raison r (avec $r > 1$) dépasse le seuil S , à l'aide d'une boucle `while`.

Exercice 7.3.2. Écrire une fonction `euclide(a, b)` qui calcule le PGCD de deux entiers a et b par l'algorithme d'Euclide (tant que b n'est pas nul, remplacer (a, b) par $(b, a \% b)$).

Exercice 7.3.3. En reprenant la méthode par balayage vue en cours (chapitre sur le T.V.I.), écrire une fonction `balayage(f, a, b, pas)` qui renvoie un encadrement d'une solution de $f(x) = 0$ sur $[a; b]$, en utilisant une boucle `while`.

Exercice 7.3.4. Écrire une fonction récursive ou itérative `fibonacci(n)` qui renvoie le n -ième terme de la suite de Fibonacci, définie par $F_0 = 0$, $F_1 = 1$ et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$ (on privilégiera une version avec boucle `for`, plus efficace qu'une version récursive naïve).

Exercice 7.3.5 (Synthèse). On considère la fonction $f(x) = x^3 - 2x - 5$. Écrire un programme complet qui :

- 1) définit la fonction `f(x)` correspondante ;
- 2) utilise une boucle `while` pour appliquer la méthode de dichotomie sur l'intervalle $[2; 3]$, avec une précision de 10^{-3} ;
- 3) affiche la valeur approchée obtenue de la solution de $f(x) = 0$.

Exercice 7.3.6 (Fonction avec liste). Écrire une fonction `maximum(liste)` qui renvoie la plus grande valeur d'une liste de nombres, **sans utiliser** la fonction native `max()` de Python (on initialisera une variable avec le premier élément, puis on parcourra la liste avec une boucle `for` en comparant chaque élément).